

前端模块化—复习要点

1 前端模块化的由来与概念

1.1 前后端分离后的前端挑战

[p.7–11]

- > 前后端分离后，前端独立开发暴露出问题 [p.7–9]:
 - 必须手动引入所有 JS 文件，且顺序很重要 [p.9]
 - 全局变量污染、命名冲突风险、函数也是全局的 [p.10]
 - JS 文件之间的依赖关系难以管理 [p.10]
- > 代码层面问题总结 [p.11]:
 - 代码质量：繁琐、可读性差、不适合复杂业务逻辑
 - 效率：一个页面可能多次异步请求，服务器压力大
 - 架构：无开发流程和构建规范，难以重用

1.2 前端工程化与模块化概念

[p.12–13]

- > 工程化 (Engineering) [p.12]: 广义上属于软件工程，包括模块化、组件化、规范化、自动化四个方面，解决可维护性、可复用性、可扩展性
- > 模块化 (Modularization) [p.12]: 将大工程拆分成相互依赖的小工程，编译或运行时再统一拼装和加载。狭义的前端工程化有时特指模块化
- > 外部模块化 [p.13]: 引入第三方包或插件，由 Node.js 和 NPM 管理
- > 内部模块化 [p.13]: 工程内部的分层或分类，由 CommonJS、ES6 Module 和构建工具组织
- > 模块源码到目标代码的编译转换：Babel 等转译工具 [p.13]

2 Node.js 与 NPM

2.1 Node.js 简介

[p.15–19]

- > Node.js 是跨平台、开源的后端 JavaScript 运行环境，运行在 Chrome V8 引擎上 [p.15]
- > Node.js = JavaScript 运行时 + 生态系统 [p.16]:
 - V8 引擎：来自 Chrome，让 JavaScript 脱离浏览器运行
 - 事件驱动、非阻塞 I/O：高性能，适合处理大量并发
 - NPM：包管理器，管理第三方依赖
 - 内置模块：fs（文件系统）、http（HTTP 服务器）、path（路径处理）
- > 对前端的意义 [p.17]:
 - 提供工程化基础：本地运行环境、包管理器、脚本工具、CommonJS 模块系统
 - 催生现代工具链：Webpack、Vite、Vue、React、Angular
 - 统一语言栈：前后端都用 JavaScript 开发

2.2 NPM 包管理器

[p.20–25]

- > **NPM** (Node Package Manager) [p.20]: Node.js 包管理工具，内置于 Node.js
- > 两大功能 [p.20]:
 - 包仓库 (Registry): 云端包存储仓库
 - **CLI 命令行工具**: 安装、卸载、管理包
- > 初始化: `npm init` → 生成 `package.json` [p.20]
- > 常用命令 [p.22]:
 - `npm install jquery` — 安装最新版到 dependencies
 - `npm install jquery@2.1.x` — 指定版本
 - `npm install --save-dev eslint` 或 `-D` — 开发依赖 (devDependencies)
 - `npm install -g webpack` — 全局安装命令行工具
- > **Yarn** [p.24]: Facebook/Google 联合推出的替代品，解决 npm 安装速度慢和版本一致性问题（并行安装、本地缓存、`yarn.lock`）

3 内部模块化的演进

3.1 原始阶段—函数作用域

[p.27–29]

- > 将 JS 文件一一引入 HTML，每个文件代表一个模块 [p.27]
- > 改进：每个模块包裹在函数作用域内执行，避免污染全局环境 [p.28]
- > 问题 [p.29]: 大量 script 标签、依赖顺序难管理、同步加载易卡死、全局变量污染

3.2 在线处理阶段—AMD/CMD

[p.30–32]

- > **AMD** (Asynchronous Module Definition): `require.js`，异步加载 [p.30]
- > **CMD** (Common Module Definition): `sea.js` [p.30]
- > 思想：提供 API 和语法声明模块及依赖关系，浏览器下载后根据声明分析依赖逐步加载（在线编译） [p.30]
- > 问题：延长页面加载时间，大量 HTTP 请求降低性能 [p.30]

3.3 预处理阶段—CommonJS / ES Module

[p.33–38]

- > 主流方案，代码部署上线前完成模块化处理 [p.33]
- > 思想：提供特殊语法，用工具（Webpack）进行代码合并，输出合并后的完整代码，减少 HTTP 请求 [p.33]
- > **CommonJS** [p.34–37]:
 - 每个文件是一个模块，有自己的作用域，变量/函数/类私有
 - `module.exports` 是对外接口，`require()` 加载模块
 - 模块首次加载后缓存，多次加载直接读缓存 [p.37]
 - 加载顺序按代码中出现的顺序 [p.37]
- > **ES Module** [p.38]:
 - ES6 之前 JS 没有自己的模块规范，社区制定 CommonJS，ES6 将其写入语言标准
 - 语法简洁优雅，所有现代浏览器默认支持
 - `export` / `import` 语法

3.4 模块化方式对比

方式	阶段	代表	特点
原始阶段	脚本引入	函数包裹	依赖顺序难管理，全局污染 [p.27-29]
AMD	在线处理	require.js	异步加载，大量 HTTP 请求 [p.30]
CMD	在线处理	sea.js	在线编译，影响性能 [p.30]
CommonJS	预处理	Node.js	module.exports/require，缓存 [p.34-37]
ES Module	预处理	ES6 标准	export/import，浏览器原生支持 [p.38]

4 构建打包工具

4.1 为什么需要构建打包工具？

[p.40–41]

- > 前端模块化目标：开发模块化而非运行模块化 [p.40]
- > 两步走 [p.40]：先通过 CommonJS/ES6 实现 JS 模块化 → 再通过 Webpack/Vite 编译处理（合并请求、去重依赖、体积优化、兼容性、导入 npm 包）
- > 构建工具三大角色 [p.41]：翻译官（语法转换）、优化师（压缩合并）、搬运工（资源处理）

4.2 Webpack

[p.42–53]

- > 前端资源加载和打包工具，根据模块依赖关系进行静态分析，按规则生成静态资源 [p.42]
- > 可将 js、css、html 等转换成一个静态文件，减少页面请求 [p.42]
- > 五大核心概念：
 - 入口 (Entry) [p.44]：指示构建依赖图的起始模块，webpack.config.js 中 entry 属性配置
 - 输出 (Output) [p.45]：指定 bundles 的输出位置和命名，output 属性配置
 - 加载器 (Loader) [p.46]：webpack 只理解 JS，非 JS 资源（CSS、图片）需 loader 处理。如 css-loader + style-loader
 - 插件 (Plugin) [p.47]：执行比 loader 范围更广的任务（打包优化、压缩、环境变量）。如 HtmlWebpackPlugin
 - 模式 (Mode) [p.48]：开发模式 vs 生产模式，后者会加密和压缩打包结果

4.3 Vite

[p.54]

- > 新一代前端构建工具，相比 Webpack 更快的开发服务器启动和热更新
- > 利用 ES Module 原生支持，开发环境无需打包

5 本章小结与考点梳理

[p.55]

- > 前端模块化概念 [p.12–13]：工程化四方面、外部/内部模块化
- > Node.js [p.15–17]：V8 引擎、事件驱动、非阻塞 I/O、对前端的三大意义
- > NPM [p.20–24]：包管理两大功能、常用命令、Yarn 改进点
- > 模块化演进 [p.27–38]：原始阶段 → 在线处理 (AMD/CMD)→ 预处理 (CommonJS/ES Module)
- > Webpack [p.42–48]：五大核心概念（Entry/Output/Loader/Plugin/Mode）

高频考点: (1) 前端工程化四方面 (模块化/组件化/规范化/自动化) [p.12]; (2) Node.js 核心组成 (V8/事件驱动/NPM/内置模块) [p.16]; (3) 前端模块化三个演进阶段及代表技术 [p.27-38]; (4) CommonJS vs ES Module 的区别 [p.34, p.38]; (5) Webpack 五大核心概念及其作用 [p.44-48]; (6) NPM 常用命令及 dependencies vs devDependencies [p.22]